CSA17 – ARTIFICIAL INTELLIGENCE

ASSIGNMENT 5 – CO5

AGENTS, INTELLIGENT SYSTEMS AND REAL-TIME AI: ARCHITECTURE, ENGINEERING AND DEPLOYMENT

CO5 Coverage: Agent architecture and its types – Swarm systems – Intelligent systems: representing and using domain knowledge, expert-system shells, explanation, knowledge acquisition – Work with PySpark and RDD – AI chatbot – Real-time emotion detection – AI-powered smart controller – AI-based intrusion detection – Voice assistant – Edge-AI systems – Disaster detection – Deepfake detection and face

authenticity verification – Autonomous drone.

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50 (Part A – 25, Part B – 25)

Submission Date: _____

Purpose of the Assignment

Part A asks you to **design** intelligent systems on paper: architectures, decision rules, pseudocode and traces. Part B asks you to **engineer** them: implement, instrument, stress, compress and defend a deployment decision. The two parts are connected: Part B implements and tests what you designed in Part A.

The central question is:

How should an AI engineer move from a working model to a deployable agent that works within limits of time, memory, data volume and risk?

PART A – DESIGN AND REASONING (25 Marks)

No compiler is to be used in Part A. All pseudocode must follow the SIMATS Engineering – Handbook for Algorithmic Pseudocode.

TASK A1 – Design an Agent Architecture (7 Marks)

Problem Context – Autonomous Parcel-Delivery Drone

A drone must deliver medical parcels across a city grid. It must avoid no-fly zones and sudden obstacles, cope with wind drift and return or land safely when the battery is low.

A1.1 – Model the Agent Problem | 1 Mark

Specify the PEAS description, classify the environment, and state which agent type(s) (simple reflex, model-based, goal-based, utility-based, learning) the problem requires and why.

A1.2 – Define Percepts, Internal State and Actions | 1 Mark

Define exactly what the drone senses, what it must remember between steps and which actions it may take.

A1.3 – Define the Decision Rule and Priorities | 1 Mark

State how conflicting goals are resolved (for example, safety over mission over battery economy) and what utility, if any, trades them off.

A1.4 – Design Complete Pseudocode | 3 Marks

Write pseudocode of the sense–decide–act cycle for a layered or hybrid architecture. It must show the reactive safety layer, the deliberative planning layer, the arbitration between them, state update and termination. Writing "plan the route appropriately" is not sufficient.

A1.5 – Trace the Agent | 1 Mark

Manually trace at least six time steps in which an obstacle suddenly appears and the battery falls below a threshold. Show percept, internal state, layer that acted and action chosen at each step.

Evidence Expected: PEAS and agent-type justification, percept/state/action definitions, priority rule, complete pseudocode and a decision trace.

TASK A2 – Don't Centralise What Can Be Local (5 Marks)

Problem Context – Flood Search-and-Rescue with 50 Drones

A control station currently assigns every drone its search cell. Communication is unreliable and drones fail during missions.

A2.1 – Diagnose the Weakness of the Centralised Design | 1 Mark

Identify what happens to coverage, communication load and mission success when the controller or links fail.

A2.2 – Redesign Using Local Rules | 2 Marks

Design a decentralised swarm strategy (for example, pheromone-like marking, frontier selection or separation/alignment/cohesion rules). Give the per-agent rule in pseudocode and state what information an agent needs from its neighbours.

A2.3 – Demonstrate the Emergent Behaviour | 1 Mark

On a 5 × 5 grid with three drones, manually trace several moves to show how the local rule produces coverage without central assignment.

A2.4 – Challenge Your Rule | 1 Mark

Construct a configuration in which your rule causes clustering, deadlock or redundant coverage. Predict the behaviour first, then trace it.

Evidence Expected: Diagnosis, per-agent rule, hand trace and the student-designed failure configuration.

TASK A3 – Discover the Expert-System Architecture Pattern (5 Marks)

Three systems are described:

1. A pump-fault diagnosis advisor (symptoms → fault)

2. A crop-disease advisor (observations → disease → treatment)

3. A flood-relief triage advisor (reports → priority)

A3.1 – Identify the Common Pattern | 2 Marks

Write a small rule set (at least four rules each) for the three domains. Identify what is common to all three and what is domain-specific. Which parts could be reused without changing any code?

A3.2 – Design a Generic Expert-System Shell | 2 Marks

Specify the components (knowledge base, working memory, inference engine, explanation facility, knowledge-acquisition interface), the data each exchanges and pseudocode of the inference cycle including conflict resolution.

A3.3 – Explain the Role of Explanation and Knowledge Acquisition | 1 Mark

Explain why the explanation facility (HOW and WHY) and the knowledge-acquisition interface are essential for a system used by domain experts who are not programmers.

Evidence Expected: Three rule sets, common-versus-specific analysis, shell design with pseudocode and the explanation/acquisition reasoning.

TASK A4 – Design a Distributed Data Pipeline with RDDs (4 Marks)

A city's air-quality agent receives about 10 million sensor readings (sensorId, timestamp, value) per day and needs the per-sensor mean and maximum, and the ten noisiest sensors.

Design the pipeline in pseudocode using RDD operations. You must identify:

• Which operations are transformations and which are actions, and when evaluation actually happens

• Which transformations are narrow and which require a shuffle

• Where caching would help and where it would waste memory

• How lineage allows recovery when a partition is lost

Writing “process the data in parallel” is not sufficient.

Evidence Expected: Complete pipeline pseudocode, narrow/wide classification, caching decision and lineage explanation.

TASK A5 – Design When the Full Model Is Too Expensive (4 Marks)

Problem Context – Detection Model on an Edge Device

A detection model (for intrusion, emotion, deepfake or disaster detection; choose one) works well on a server but must run on a device with limited memory and a latency budget of 50 ms per sample.

A5.1 – Full-Model Perspective | 1 Mark

Estimate the parameters, memory and operations per sample of your full model and state why it exceeds the budget.

A5.2 – Design the Compressed Pipeline | 2 Marks

Design an approximation (for example, pruning, quantisation, a smaller architecture, feature reduction or an early-exit cascade). Give pseudocode of the inference pipeline and state which operations are saved.

A5.3 – What Have We Sacrificed? | 1 Mark

Identify precisely what accuracy, robustness or explainability may be lost and how you would detect it.

Evidence Expected: Resource estimate, compressed-pipeline pseudocode and an analysis of the trade off.

PART B – IMPLEMENTATION, INSTRUMENTATION AND DEPLOYMENT DECISIONS (25 Marks)

Python is permitted in Part B. For every task submit source code, test input, expected behaviour, actual output, performance metrics and interpretation. Do not stop when the program runs.

TASK B1 – Implement and Instrument the Drone Agent (7 Marks)

Implement your Task A1 architecture and Task A2 swarm rule in a grid simulator.

B1.1 – Validate the Implementation | 2 Marks

Test at least three scenarios: normal flight, a sudden obstacle, and a boundary case (battery exhausted or goal blocked). State what each scenario verifies and show the behaviour log.

B1.2 – Compare Design with Implementation | 1 Mark

Compare your Task A1.5 hand trace with the program's trace and explain every difference. **B1.3 – Improve the Design | 2 Marks**

Introduce swarm coordination for *N* drones using your Task A2 rule (or an improved reactive/deliberative arbitration). State what the improvement targets.

B1.4 – Measure the Effect | 2 Marks

Over at least 30 seeds, compare single-drone and swarm operation on mission success rate, coverage percentage against time, redundancy ratio, battery used, decision latency and robustness when 0%, 10% and 30% of the drones fail. Test *N* = 5, 10, 25 and 50.

Evidence Expected: Simulator and agent code, scenario logs, trace comparison, before/after metric tables and graphs, and an interpretation of the scaling behaviour.

TASK B2 – Engineer an Adversarial Input for the Expert System (5 Marks)

Implement your Task A3 shell with a pump-fault rule base (at least 15 rules and 5 fault types) with HOW and WHY explanations.

B2.1 – Implement and Evaluate | 2 Marks

Write the rules using only 15 of 30 expert-labelled cases and keep the other 15 as a hold-out set. Report diagnosis accuracy, per-fault precision and recall, rules fired per case and sample explanations.

B2.2 – Engineer Adversarial Cases | 2 Marks

Design inputs that expose weaknesses: conflicting rules, missing evidence and dependence on rule order. Predict the outcome of each before running, then compare at least two conflict-resolution strategies (specificity, priority or recency).

B2.3 – Knowledge Acquisition Test | 1 Mark

Add or edit a rule at run time without changing engine code and show, with a regression run, which earlier diagnoses were affected.

Evidence Expected: Shell code, hold-out results, adversarial case table with predictions and outcomes, strategy comparison and the regression result.

TASK B3 – Applied Case: Full Model versus Edge Model (8 Marks)

Choose **one** applied case and one public dataset or simulator (cite the source): **(a)** AI-based intrusion detection, **(b)** real-time emotion detection, **(c)** deepfake detection and face-authenticity verification, **(d)** disaster detection, **(e)** AI-powered smart controller, **(f)** another edge-AI system approved by the instructor. Use the case chosen in Task A5.

B3.1 – Correctness on a Small Benchmark | 2 Marks

Build the full model with a leakage-safe split (time-ordered for intrusion data; subject-wise or source-wise for emotion and deepfake data). Report accuracy, precision, recall, F1, ROC-AUC or PR-AUC and the confusion matrix. For case (e) report settling time, overshoot, energy use and constraint violations against a rule-based baseline.

B3.2 – Find a Counterexample | 2 Marks

Find and categorise at least five mis-predictions. Identify a class of inputs on which the model fails (for example, an unseen attack type, unseen generator, poor lighting or sensor noise) and demonstrate it.

B3.3 – Increase the Problem Size | 2 Marks

Using the Task A4 RDD pipeline in PySpark, process 10^5, 10^6 and 10^7 records (feature extraction or batch scoring). Record runtime against partitions {1, 2, 4, 8}, the effect of caching and the number of shuffles. Explain any case where more partitions did not improve speed.

B3.4 – Engineering Trade-Off | 2 Marks

Implement the Task A5 compressed model. Report latency per sample (mean and 95th percentile), throughput, model size and the accuracy change against the full model. State whether the 50 ms budget is met.

Evidence Expected: Code, split description, metric tables, error categories, scaling table, compression comparison and interpretation.

TASK B4 – Final Engineering Decision: Conversational Interface and Deployment (5 Marks)

Operators want to ask the Task B3 system questions through a chatbot or voice assistant (for example, “Which alerts are open?”, “Why was this flagged?”).

B4.1 – Implement a Practical Strategy | 2 Marks

Build a retrieval-based chatbot with at least 8 intents (TF-IDF with cosine similarity or a classifier) and an out-of-scope fallback controlled by a confidence threshold. Report per-intent precision, recall and F1, confusion matrix and response latency.

B4.2 – Challenge Your Strategy | 1 Mark

Simulate speech-recognition errors by corrupting test utterances at word-error rates of 0%, 10%, 20% and 30%. Report intent accuracy, out-of-scope detection and false-accept rate, and show a threshold-sweep curve.

B4.3 – Production Decision | 2 Marks

Decide whether the detection model and the assistant should run on the edge device or in the cloud. Defend the decision using your own latency, accuracy, throughput,

privacy and cost evidence, and state the conditions under which you would reverse it.

Evidence Expected: Chatbot code, intent-level metrics, word-error-rate degradation table, threshold curve and an evidence-based deployment decision.

MARK DISTRIBUTION – ASSIGNMENT 5

Part	Task	Assessed Area	Marks
A	A1	Agent architecture design	7
A	A2	Swarm design	5
A	A3	Expert-system architecture	5
A	A4	RDD pipeline design	4
A	A5	Edge-model design	4
		Part A Total	**25**

Part	Task	Assessed Area	Marks
B	B1	Drone agent implementation and measurement	7
B	B2	Expert-system engineering and adversarial testing	5
B	B3	Applied case: full versus edge model	8
B	B4	Conversational interface and deployment decision	5
		Part B Total	**25**
		TOTAL	**50**

SUBMISSION REQUIREMENTS

Part A: Problem formulation, agent/system definition, pseudocode, manual traces, design reasoning.

Part B: Source Code → Test Input → Expected Behaviour → Actual Output → Performance Metrics (latency, throughput, model size, accuracy, F1 and others as applicable) → Boundary/Failure Test → Interpretation.

Screenshots containing only a final output such as “Accuracy = 0.96” will not constitute sufficient evidence.

SOFTWARE ENGINEERING EXPECTATIONS

Meaningful names, modular design, separation of agent logic from simulation and input/output, handling of boundary conditions, clear assumptions, fixed random seeds and reproducible test cases. A program that produces the expected output but whose logic cannot be explained or validated will not receive full credit.

IMPORTANT RULE

DO NOT STOP WHEN THE PROGRAM RUNS. Establish: *Does the system work? → How many resources does it need and why? → When does it stop being practical? → What should the engineer do about it?*

ENGINEERING WORKFLOW: UNDERSTAND → DESIGN → ARCHITECT → IMPLEMENT → MEASURE → STRESS → COMPRESS → DECIDE → DEFEND

Assessment Verification: Students may be asked to give a short live demonstration and to explain any design, code, dataset split, metric or deployment decision in a brief viva.

CSA17 – Artificial Intelligence

Assignment 5 – CO5

Title: Agents, Intelligent Systems and Real-Time AI: Architecture, Engineering and Deployment

PART A – DESIGN AND REASONING (25 Marks)

TASK A1 – Design an Agent Architecture (7 Marks)

A1.1 – Model the Agent Problem

PEAS description

PEAS stands for Performance measure, Environment, Actuators and Sensors.

Component	Description
Performance measure	Safe delivery, shortest practical route, low battery usage, obstacle avoidance and successful landing
Environment	City roads, buildings, delivery locations, no-fly zones, wind and moving obstacles
Actuators	Motors, propellers, steering, altitude control, landing gear and communication system
Sensors	GPS, camera, LiDAR, ultrasonic sensor, battery sensor, gyroscope and wind sensor

Environment classification

Property	Classification
Observability	Partially observable
Agents	Dynamic environment with other moving agents
Deterministic	Stochastic because wind and obstacles can change
Episodic/Sequential	Sequential
Static/Dynamic	Dynamic
State	Continuous position, speed, altitude and battery level
Known/Unknown	Partially unknown

Required agent types

1. Model-based agent: Remembers its position, battery level, obstacles and previous route.
2. Goal-based agent: Works towards the goal of delivering the medical parcel.
3. Utility-based agent: Selects a safe and efficient route by considering time, battery and risk.
4. Learning agent: Learns from previous flights and improves route selection.
5. Simple reflex component: Reacts immediately to nearby obstacles and emergency conditions.

Conclusion: A hybrid agent combining model-based, goal-based, utility-based and learning behaviour with a simple reactive safety layer is suitable for the drone.

A1.2 – Define Percepts, Internal State and Actions

1. Percepts – What the drone senses

- Current GPS location.
- Destination location.
- Nearby obstacles.
- No-fly zone information.
- Battery percentage.
- Wind speed and direction.
- Speed and altitude.
- Parcel delivery status.

2. Internal state – What the drone remembers

- Current position and previous position.
- Destination coordinates.
- Planned route.
- Detected obstacle locations.
- Remaining battery percentage.
- Visited locations.
- Current flight mode: normal, avoiding, returning or landing.
- Delivery status.

3. Actions – What the drone can do

- Move forward, backward, left and right.
- Increase or decrease altitude.
- Change speed.
- Stop or hover.
- Avoid an obstacle.
- Recalculate the route.
- Deliver the parcel.
- Return to the charging station.
- Land safely.

A1.3 – Define the Decision Rule and Priorities

The drone follows this priority order:

1. Safety: Avoid obstacles and no-fly zones.
2. Battery protection: Return or land safely when battery is low.
3. Mission completion: Deliver the medical parcel.
4. Efficiency: Select a shorter route and reduce energy consumption.

A utility function can be used to compare safe routes:

[U = 50D + 30S + 15B + 5E]

Where:

- (D) = delivery progress.
- (S) = safety score.
- (B) = battery efficiency.
- (E) = route efficiency.

Safety and emergency rules have higher priority than this utility calculation. The drone must never select a route through a no-fly zone simply because it is shorter.

A1.4 – Complete Pseudocode

Hybrid drone agent

BEGIN

INITIALISE drone position

INITIALISE destination

INITIALISE battery level

INITIALISE internal state

LOAD no-fly zone map

SET mode = NORMAL

WHILE drone is not landed

SENSE current position

SENSE nearby obstacles

SENSE wind direction

SENSE battery level

```
UPDATE internal state
IF obstacle is very close THEN
    SET mode = AVOID
    STOP forward movement
    SELECT safe direction
    MOVE away from obstacle
ELSE IF battery level <= 20% THEN
    SET mode = RETURN
    FIND nearest safe landing station
    PLAN route to landing station
    MOVE towards landing station
ELSE IF destination is reached THEN
    SET mode = DELIVER
    RELEASE medical parcel
    SET delivery status = COMPLETE
    FIND safe landing location
    LAND drone
ELSE
    SET mode = NORMAL
    GENERATE possible routes
    REMOVE routes through no-fly zones
    REMOVE routes with dangerous obstacles
    CALCULATE route distance
    CALCULATE battery cost
    CALCULATE route safety
    SELECT route with highest utility
    MOVE one step along selected route
END IF
UPDATE position
```

UPDATE battery level

SAVE current state

END WHILE

DISPLAY delivery status

DISPLAY final battery level

END

Explanation: The reactive layer handles immediate dangers. The planning layer calculates a safe route. The arbitration rule gives emergency safety and low-battery conditions priority over normal delivery.

A1.5 – Manual Trace of the Agent

Assume:

- Starting position = (0,0).
- Destination = (3,3).
- Initial battery = 100%.
- Low-battery threshold = 20%.
- An obstacle suddenly appears at (2,1).

Step	Percept	Internal state	Layer acting	Action
1	No obstacle, battery 100%	Position (0,0), destination (3,3)	Planning	Move to (1,0)
2	No obstacle, battery 90%	Position (1,0)	Planning	Move to (2,0)
3	Obstacle detected at (2,1)	Position (2,0), obstacle stored	Reactive safety	Hover and avoid obstacle
4	Obstacle still nearby, battery 60%	Position (2,0), safe route required	Planning + safety	Move to (3,0)
5	Battery falls to 18%	Position (3,0), battery below threshold	Safety arbitration	Cancel delivery route and return

6	Safe landing point detected	Position (2,0), return mode	Planning + safety	Move to safe landing point and land

Result: The drone avoids the obstacle and gives battery protection priority when the battery falls below 20%.

TASK A2 – Don't Centralise What Can Be Local (5 Marks)

A2.1 – Weaknesses of Centralised Design

In a centralised system, one control station assigns search areas to all 50 drones.

Its weaknesses are:

1. Controller failure: If the central controller stops working, drones may not receive new instructions.
2. Communication failure: Drones may lose their assigned search areas.
3. High communication load: Every drone continuously sends information to the controller.
4. Poor scalability: Managing more drones increases the controller's workload.
5. Mission failure: If some drones fail, uncovered areas may remain unsearched.

Conclusion: A decentralised swarm system is better because every drone can make local decisions and continue searching without depending on one central controller.

A2.2 – Decentralised Swarm Strategy

Proposed method: Local frontier selection with virtual pheromone marking

Each drone selects the nearest unvisited search cell. When it visits a cell, it marks that cell as visited. Nearby drones share their positions and visited-cell information.

Information required from neighbours:

- Neighbouring drone positions.
- Visited cells.
- Current direction.
- Drone status.
- Nearby obstacles.

Per-agent pseudocode

BEGIN

FOR EACH drone

```
    SENSE nearby cells
    SENSE neighbouring drones
    RECEIVE visited-cell information
    MARK current cell as visited
    FIND all safe unvisited neighbouring cells
    IF unvisited cells are available THEN
        SELECT cell with lowest travel cost
        AVOID cells occupied by other drones
        MOVE to selected cell
    ELSE
        SEARCH for nearest unvisited frontier
        IF frontier exists THEN
            MOVE towards frontier
        ELSE
            STOP searching
        END IF
    END IF

    SHARE visited-cell information
    UPDATE drone position
END FOR
END
```

Advantages:

- No central controller is required.
- Drones can continue working during communication failures.
- Search areas are distributed automatically.
- Failed drones do not stop the entire swarm.

A2.3 – Emergent Behaviour on a 5 × 5 Grid

Assume three drones start at:

- Drone D1 = (1,1)
- Drone D2 = (3,1)
- Drone D3 = (2,3)

The grid contains 25 cells. Each drone chooses an unvisited neighbouring cell.

Step	Drone D1	Drone D2	Drone D3	Behaviour
1	(1,1)	(3,1)	(2,3)	Initial positions
2	(1,2)	(3,2)	(2,2)	Search begins
3	(0,2)	(4,2)	(2,4)	Drones move towards different areas
4	(0,3)	(4,3)	(1,4)	More cells are covered
5	(0,4)	(4,4)	(1,3)	Coverage expands

Result: The drones collectively explore different regions using local rules without central assignment.

A2.4 – Challenge the Rule

Failure configuration: Clustering

Suppose all three drones start close together:

- D1 = (2,2)
- D2 = (2,3)
- D3 = (3,2)

If all drones select the nearest unvisited cell without considering other drones' destinations, they may repeatedly choose the same area.

Predicted behaviour: Drones cluster together and repeatedly search nearby cells, reducing total coverage.

Step	D1	D2	D3	Problem
1	(2,2)	(2,3)	(3,2)	Drones are close
2	(1,2)	(1,2)	(1,2)	Same cell selected
3	(1,3)	(1,3)	(1,3)	Redundant movement
4	(0,3)	(0,3)	(0,3)	Coverage remains low

Improvement: Add a separation rule. Each drone should avoid a cell already selected by a neighbour and prefer a different unvisited frontier.

TASK A3 – Discover the Expert-System Architecture Pattern (5 Marks)

A3.1 – Common Pattern and Rule Sets

An expert system uses a collection of IF–THEN rules to solve problems in a specific domain.

1. Pump-fault diagnosis advisor

Rule	IF condition	THEN conclusion
P1	Pump makes unusual noise	Possible bearing fault
P2	Motor becomes very hot	Possible motor overheating
P3	Water flow is low and filter is blocked	Filter blockage
P4	Pump vibrates heavily	Possible misalignment
P5	Pressure is low and pipe leaks	Pipe leakage

2. Crop-disease advisor

Rule	IF condition	THEN conclusion
C1	Leaves have yellow spots	Possible leaf disease
C2	Leaves have white powder	Possible powdery mildew
C3	Roots are damaged and soil is wet	Possible root rot
C4	Leaves are dry and soil is dry	Water stress
C5	Brown patches appear on leaves	Possible fungal infection

3. Flood-relief triage advisor

Rule	IF condition	THEN conclusion
F1	Person is injured and trapped	Critical priority
F2	Person is elderly and isolated	High priority
F3	Family has drinking water but needs food	Medium priority
F4	Person has food, water and shelter	Low priority

F5	Person is missing in a flooded area	Critical priority

Common features

- All three systems use a knowledge base.
- All three accept facts or observations.
- All three use IF–THEN rules.
- All three use an inference engine.
- All three produce a recommendation or decision.

Domain-specific features

System	**Domain-specific knowledge**
Pump advisor	Noise, pressure, temperature and vibration
Crop advisor	Leaf colour, spots, roots and soil condition
Flood advisor	Injuries, shelter, food and rescue urgency

Reusable components: The inference engine, working memory, explanation facility and knowledge-acquisition interface can be reused without changing their core code. Only the domain-specific rules and input facts need to change.

A3.2 – Generic Expert-System Shell

Components

Component	**Function**
Knowledge base	Stores IF–THEN rules
Working memory	Stores current facts and observations
Inference engine	Matches facts with rules and derives conclusions
Explanation facility	Explains why and how a conclusion was reached

Knowledge-acquisition interface Allows experts to add, edit or remove rules

Data exchange

1. User provides observations.
2. Observations are stored in working memory.
3. Inference engine checks rules in the knowledge base.
4. Matching rules are fired.

5. New facts and conclusions are added to working memory.
6. Explanation facility records the rules used.
7. Final result is displayed to the user.

Inference-cycle pseudocode

```
BEGIN
LOAD knowledge base
READ user observations
STORE observations in working memory
REPEAT
    MATCH rules with working memory
    CREATE list of applicable rules
    IF no applicable rules exist THEN
        EXIT LOOP
    END IF
    SELECT highest-priority rule
    FIRE selected rule
    ADD new conclusion to working memory
    RECORD fired rule for explanation
UNTIL no new facts are produced
DISPLAY final conclusion
DISPLAY explanation of rules used
END
```

Conflict resolution: When multiple rules match, select the rule with the highest priority. If priorities are equal, select the more specific rule containing more conditions.

A3.3 – Role of Explanation and Knowledge Acquisition

Explanation facility

The explanation facility answers two questions:

- WHY: Why did the system reach this conclusion?
- HOW: How did the system reach this conclusion?

Example:

Result: Pump bearing fault.

WHY: The pump produced unusual noise and heavy vibration.

HOW: Rule P1 and Rule P4 matched the observations and produced the diagnosis.

This improves transparency and helps domain experts trust and verify the system.

Knowledge-acquisition interface

It allows domain experts to add, update or remove rules without changing the program code.

For example, a pump engineer can add a new rule for a newly discovered pump fault.

Conclusion: Explanation improves trust, while knowledge acquisition keeps the expert system updated.

TASK A4 – Design a Distributed Data Pipeline with RDDs (4 Marks)

The city receives 10 million sensor readings per day. Each reading contains:

(sensorId, timestamp, value)

The system must calculate the mean and maximum value for every sensor and identify the ten noisiest sensors.

RDD pipeline pseudocode

BEGIN

READ sensor data from input file as RDD

FILTER invalid readings

MAP each reading into:

    (sensorId, (value, value, 1))

REDUCE BY KEY:

    total value = sum of values

    maximum value = maximum reading

    count = sum of counts

CALCULATE:

    mean = total value / count

CREATE result:

(sensorId, mean, maximum value)

SORT sensors by maximum value in descending order

SELECT first 10 sensors

DISPLAY per-sensor mean and maximum

DISPLAY ten noisiest sensors

END

PySpark RDD implementation

from pyspark import SparkContextsc = SparkContext("local[*]", "AirQuality")data = sc.parallelize([("S1", 10), ("S2", 30), ("S1", 20), ("S2", 50), ("S3", 40), ("S1", 30), ("S3", 60)])pairs = data.map(lambda x: (x[0], (x[1], x[1], 1)))result = pairs.reduceByKey(lambda a, b: (a[0] + b[0], max(a[1], b[1]), a[2] + b[2]))summary = result.mapValues(lambda x: (x[0] / x[2], x[1]))print("Sensor Mean and Maximum:")for item in summary.collect(): print(item)print("Top 10 Noisiest Sensors:")top10 = summary.takeOrdered(

Note: This is a small demonstration dataset. In the actual assignment, replace it with the 10-million-record sensor dataset.

Transformations and Actions

Operation	Type	Explanation
map()	Transformation	Converts each record into a key-value pair
filter()	Transformation	Removes invalid readings
reduceByKey()	Transformation	Combines values belonging to the same sensor
mapValues()	Transformation	Calculates mean and maximum
takeOrdered()	Action	Returns the ten highest maximum readings
collect()	Action	Brings results to the driver program

Spark uses lazy evaluation. Transformations are not executed immediately. Execution starts when an action such as collect() or takeOrdered() is called.

Narrow and Wide Transformations

Transformation	Type	Reason
map()	Narrow	Each output depends on one input partition

filter()	Narrow	Filters records within the same partition
mapValues()	Narrow	Processes values without redistributing keys
reduceByKey()	Wide	Requires combining matching keys across partitions
sortByKey()	Wide	Requires data redistribution for sorting

Caching

Caching is useful when the same processed RDD is reused multiple times.

For example, the cleaned sensor readings can be cached if several calculations use them.

Caching every intermediate RDD wastes memory. It is better to cache only frequently reused RDDs.

Lineage and Fault Recovery

RDD lineage records the sequence of transformations used to create an RDD.

If a partition is lost, Spark can recreate that partition by repeating the required transformations from the original data.

Conclusion: RDDs process large sensor datasets in parallel. Transformations prepare the data, actions execute the computation, and lineage helps recover lost partitions.

TASK A5 – Design When the Full Model Is Too Expensive (4 Marks)

Selected application: AI-based intrusion detection

An intrusion detection system identifies whether network traffic is normal or malicious.

A5.1 – Full-Model Perspective

Assume the server model is a fully connected neural network:

- Input features = 100.
- Hidden layer 1 = 256 neurons.
- Hidden layer 2 = 128 neurons.
- Output layer = 2 neurons.

Parameter calculation

Layer	**Parameters**

Input to hidden layer 1	(100 times 256 + 256 = 25,856)
Hidden layer 1 to hidden layer 2	(256 times 128 + 128 = 32,896)
Hidden layer 2 to output	(128 times 2 + 2 = 258)
Total	59,010

Assuming 32-bit floating-point parameters:

[Memory = 59010 times 4 = 236040 bytes]

[Memory approx 0.225 MB]

The parameter memory is small, but the full system also requires memory for input data, intermediate activations, runtime libraries and other processes.

Approximate forward-pass multiply operations:

[100 times 256 + 256 times 128 + 128 times 2 = 58,880]

This model may still exceed the 50 ms target on a slow edge device due to preprocessing, software overhead and limited hardware. The actual latency must be measured.

A5.2 – Compressed Pipeline

Use the following techniques:

1. Reduce the network size.
2. Apply 8-bit quantisation.
3. Remove unnecessary input features.
4. Use an early decision for clearly normal or malicious traffic.

Compressed model

- Input features = 50.
- Hidden layer = 64 neurons.
- Output layer = 2 neurons.
- Quantisation = 8-bit.

Pseudocode

BEGIN

READ network traffic

EXTRACT important 50 features

```
NORMALISE input features
CONVERT values to 8-bit format
RUN compressed neural network
IF confidence >= 0.90 THEN
    DISPLAY prediction
ELSE
    RUN detailed verification model
END IF
IF prediction is malicious THEN
    GENERATE security alert
ELSE
    MARK traffic as normal
END IF
END
```

Operations saved:

- Fewer input features reduce calculations.
- Fewer neurons reduce the number of parameters.
- Quantisation reduces model storage and can speed up inference on suitable hardware.
- Early exit avoids unnecessary detailed computation for confident predictions.

A5.3 – What Have We Sacrificed?

The compressed model may have:

1. Lower detection accuracy.
2. More false negatives for unusual attacks.
3. Reduced ability to recognise new attack patterns.
4. Lower confidence under noisy or unfamiliar network conditions.

How to detect these problems:

- Compare full-model and compressed-model accuracy.
- Measure precision, recall and F1-score.

- Check the confusion matrix.
- Test unseen attack types.
- Measure mean and 95th-percentile latency.
- Test whether the model meets the 50 ms target.

Conclusion: A compressed model uses fewer resources and is more suitable for edge deployment, but its detection quality must be tested before production use.

PART B – IMPLEMENTATION, INSTRUMENTATION AND DEPLOYMENT DECISIONS (25 Marks)

TASK B1 – Implement and Instrument the Drone Agent (7 Marks)

B1.1 – Validate the Implementation

The drone simulator must be tested in three situations.

Scenario	Test input	Expected behaviour
Normal flight	No obstacles and battery above 20%	Drone moves towards destination
Sudden obstacle	Obstacle appears in the next cell	Drone stops and selects another safe cell
Boundary case	Battery reaches 10%	Drone cancels delivery and returns or lands safely

Python implementation

This simple grid simulator demonstrates the reactive safety and planning behaviour.

```
import randomSIZE = 5START = (0, 0)GOAL = (4, 4)def drone(obstacles, battery): position = START visited = [position] while position != GOAL and battery > 20: x, y = position moves = [ (x + 1, y), (x, y + 1), (x - 1, y), (x, y - 1) ] safe = [ p for p in moves if 0 <= p[0] < SIZE and 0 <= p[1] < SIZE and p not in obstacles and p not in visited ] if not safe: print("No safe route. Landing.") return position = safe[0] visited.append(position) battery -= 10 print("Position:", position, "Battery:", battery)
```

Test input: The grid is 5 × 5, start is (0,0), destination is (4,4), and obstacles are at (2,0) and (2,1).

Expected behaviour: The drone avoids blocked cells and moves through available cells. It stops when it reaches the goal, runs out of safe moves, or reaches the battery threshold.

This is a simplified demonstration. It does not yet implement GPS, wind drift, real battery physics or a physical drone controller.

B1.2 – Compare Design with Implementation

Design trace	Program behaviour
Drone senses obstacles	Program checks the obstacle set
Reactive layer avoids obstacles	Program removes blocked cells from possible moves
Planning layer selects a route	Program selects the first available safe move
Low battery triggers return	Program stops and displays a low-battery message

Difference: The manual design uses separate reactive and planning layers. The demonstration program combines these decisions in one function. A complete implementation should separate them into independent functions.

B1.3 – Improve the Design Using Swarm Coordination

The improved system uses multiple drones to search different cells.

Each drone shares its visited cells with neighbouring drones. A drone selects a safe cell that has not been visited by another drone.

Improvement targets:

- Increase area coverage.
- Reduce repeated searches.
- Continue searching when one drone fails.
- Reduce dependence on the central controller.

B1.4 – Measure the Effect

Run at least 30 different random seeds for each drone count.

Test:

[N = 5, 10, 25, 50]

Also test drone failure rates of 0%, 10% and 30%.

Metrics

Metric	Formula / meaning
Mission success rate	Successful missions / Total missions × 100

Coverage percentage	Visited cells / Total search cells × 100
Redundancy ratio	Repeated visits / Total visits
Battery used	Initial battery − Final battery
Decision latency	Time taken to choose an action
Robustness	Mission performance when drones fail

Result table format

Drones	Failure rate	Success rate	Coverage	Redundancy	Battery used	Latency
5	0%	To measure	To measure	To measure	To measure	To measure
10	10%	To measure	To measure	To measure	To measure	To measure
25	30%	To measure	To measure	To measure	To measure	To measure
50	0%	To measure	To measure	To measure	To measure	To measure

Interpretation: Increasing the number of drones can improve coverage, but communication overhead and repeated visits may reduce efficiency. The best swarm size is the one that provides good coverage with acceptable battery usage and latency.

TASK B2 – Engineer an Adversarial Input for the Expert System (5 Marks)

B2.1 – Implement and Evaluate

The pump-fault expert system uses 15 rules and five fault types.

Five fault types

1. Bearing fault.
2. Motor overheating.
3. Filter blockage.
4. Shaft misalignment.
5. Pipe leakage.

Python program – Expert-system shell

rules = [({"noise", "vibration"}, "Bearing fault", 5), ({"noise", "heat"}, "Bearing fault", 3), ({"heat", "high_current"}, "Motor overheating", 5), ({"heat", "slow_motor"}, "Motor overheating", 4), ({"low_flow", "blocked_filter"}, "Filter blockage", 5), ({"low_flow", "dirty_filter"}, "Filter blockage", 4), ({"vibration", "uneven_base"}, "Shaft misalignment", 5), ({"vibration", "shaft_noise"}, "Shaft misalignment", 4), ({"low_pressure", "pipe_leak"}, "Pipe leakage", 5), ({"low_pressure", "water_loss"}, "Pipe leakage", 4), ({"noise", "high_current"}, "Bearing fault", 2), ({"low_flow", "low_pressure"}, "Filter blockage", 2), ({"heat", "burning_smell"}, "Motor overheating", 5), ({"vibration", "loose_coupling"}, "Shaft misalignment", 5), ({"water_loss", "wet_floor"}, "Pipe leakage", 5)]def diagnose(symptoms): matched = [] for condition, fault, priority in rules: if condition.issubset(symptoms): matched.append((len(condition), priority, fault, condition)) if not matched: return "Unknown fault", [] matched.sort(reverse=True) best = matched[0] explanation = best[3] return best[2], explanation

Sample test input

noise,vibration

Expected behaviour

PUMP FAULT DIAGNOSIS

Diagnosis: Bearing fault

WHY: {'noise', 'vibration'}

HOW: Matching rules were applied.

The system checks the symptoms against the knowledge base and selects the matching rule with the highest specificity and priority.

Dataset evaluation method

The assignment requires 30 expert-labelled cases:

- Training set: First 15 cases.
- Hold-out set: Remaining 15 cases.
- The rules must be developed using only the training cases.
- The hold-out cases must be used only for evaluation.

Use the following formulas:

[Accuracy = frac{Correct predictions}{Total predictions}]

[Precision = frac{TP}{TP+FP}]

[Recall = frac{TP}{TP+FN}]

[F1 = frac{2 times Precision times Recall}{Precision+Recall}]

Important: The 15 rules above are a demonstration knowledge base. They are not proof of accuracy on the required 30 expert-labelled cases. Actual accuracy, precision and recall must be calculated using the expert-labelled data.

B2.2 – Engineer Adversarial Cases

Adversarial cases are inputs designed to expose weaknesses in the expert system.

Test case	**Input**	**Predicted problem**	**Possible result**
Conflicting rules	noise, vibration, high_current	More than one rule may match	Bearing diagnosis may compete with other diagnoses
Missing evidence	heat only	Not enough symptoms	Unknown fault
Rule-order dependence	low_flow, low_pressure, blocked_filter	Multiple rules may match	Different results if first-match strategy is used
Unusual symptoms	smoke, smell	No matching rule	Unknown fault

Conflict-resolution strategies

Strategy	**Working method**
Specificity	Selects the rule with the greatest number of matching conditions
Priority	Selects the rule with the highest assigned priority
Recency	Selects the rule that uses the most recently added fact

Comparison: First-match selection is sensitive to rule order. Specificity and priority provide more controlled decisions. A practical system can use specificity first and priority second.

B2.3 – Knowledge Acquisition Test

Knowledge acquisition allows an expert to add a rule without modifying the inference engine.

Add the following rule:

rules.append(({"smoke", "burning_smell"}, "Motor overheating", 6))

Test input before adding the rule

smoke,burning_smell

Expected result: Unknown fault

Test input after adding the rule

smoke,burning_smell

Expected result: Motor overheating

Regression test: Run the previous test cases again after adding the new rule. Check whether their diagnoses remain unchanged.

Conclusion: The knowledge base can be updated independently of the inference engine. This makes the expert system easier to maintain.

TASK B3 – Applied Case: Full Model versus Edge Model (8 Marks)

Selected case: AI-based intrusion detection.

Purpose: Detect whether network traffic is normal or represents a possible cyberattack.

B3.1 – Correctness on a Small Benchmark

Use the UNSW-NB15 network intrusion dataset as the selected public dataset. The dataset was developed by the Australian Centre for Cyber Security at UNSW Canberra.

The data contains normal network traffic and different attack categories.

Leakage-safe split

For intrusion detection, use a time-ordered split:

- 70% earlier records for training.
- 15% later records for validation.
- 15% latest records for testing.

Do not randomly mix future records into the training set when evaluating performance on later traffic.

Python full-model implementation

The following example uses a Random Forest classifier. It assumes a cleaned CSV file with a binary label column, where 0 means normal and 1 means attack.

import pandas as pdfrom sklearn.ensemble import RandomForestClassifierfrom sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, roc_auc_score)df = pd.read_csv("intrusion.csv")X = df.drop("label", axis=1)y = df["label"]# Keep numeric features for this exampleX = X.select_dtypes(include="number").fillna(0)# Time-ordered splitn = len(df)a = int(n *

0.70)b = int(n * 0.85)X_train, X_test = X.iloc[:a], X.iloc[b:]y_train, y_test = y.iloc[:a], y.iloc[b:]model = RandomForestClassifier(n_estimators=100, random_state=42)model.fit(X_train, y_train)pred = model.predict(X_test)prob = model.predict_proba(X_test)[:, 1]print("Accuracy:", accuracy_score(y_test, pred))print("Precision:", precision_score(y_test, pred))

Expected behaviour

- The model learns patterns from the training network traffic.
- It predicts normal and attack traffic in the test set.
- The evaluation reports accuracy, precision, recall, F1, ROC-AUC and the confusion matrix.

Metrics interpretation

- Accuracy measures the overall number of correct predictions.
- Precision measures how many predicted attacks are actually attacks.
- Recall measures how many actual attacks are detected.
- F1 balances precision and recall.
- ROC-AUC measures the model's ability to distinguish the two classes.

Dataset note: The example assumes that the CSV is already cleaned, numeric features are prepared, and the label column is correctly mapped to 0 and 1. For a strict time-based evaluation, use the dataset's actual timestamps or official chronological ordering rather than assuming the CSV rows are already chronological.

B3.2 – Find a Counterexample

The following five situations can cause incorrect predictions.

No.	Counterexample	Possible model error	Reason
1	New attack type	False negative	Model has not learned the new attack pattern
2	Very small attack traffic	False negative	Attack resembles normal traffic
3	Unusual network packet size	False positive	Unfamiliar traffic pattern

4	Encrypted or hidden traffic	False negative	Important features may not be visible
5	Noisy or incomplete sensor records	False positive or false negative	Incorrect input features

Failure analysis: The model may fail when it receives data that is different from the training data. This is called a distribution shift.

Improvement:

- Collect newer network traffic.
- Retrain using new attack patterns.
- Monitor false negatives.
- Use human verification for high-risk alerts.
- Test the model using unseen attack categories.

B3.3 – Increase the Problem Size Using PySpark

The system must process:

[10^5,quad 10^6,quad 10^7]

records using 1, 2, 4 and 8 partitions.

PySpark implementation

from pyspark import SparkContextimport timesc = SparkContext("local[*]", "IntrusionRDD")for size in [10**5, 10**6, 10**7]: for partitions in [1, 2, 4, 8]: data = sc.parallelize(range(size), partitions) start = time.time() features = data.map(lambda x: (x % 100, 1)) result = features.reduceByKey(lambda a, b: a + b) output = result.collect() end = time.time() print("Records:", size, "Partitions:", partitions, "Time:", round(end - start, 3), "Seconds")sc.stop()

Expected behaviour

The program creates different dataset sizes, changes the partition count and calculates grouped values.

Result table

Records	**Partitions**	**Runtime**	**Shuffles**
100,000	1	To measure	1
100,000	2	To measure	1

100,000	4	To measure	1
100,000	8	To measure	1
1,000,000	1	To measure	1
1,000,000	2	To measure	1
1,000,000	4	To measure	1
1,000,000	8	To measure	1
10,000,000	1	To measure	1
10,000,000	2	To measure	1
10,000,000	4	To measure	1
10,000,000	8	To measure	1

Caching experiment

Caching is useful if the same RDD is reused for multiple operations.

Example:

data.cache()

Caching may reduce repeated computation, but it consumes memory. It may not help when an RDD is used only once.

Why more partitions may not improve speed

- Small datasets have little work to distribute.
- Too many partitions create task-scheduling overhead.
- Limited CPU cores restrict parallel execution.
- Shuffle operations require data movement.
- Memory and disk limitations can slow processing.

Conclusion: Partitioning can improve large-scale processing, but the best partition count depends on the available hardware and data size.

B3.4 – Engineering Trade-Off

The full model is a Random Forest with 100 trees. For edge deployment, use a smaller Random Forest with 20 trees and limit the tree depth.

Compressed model code

from sklearn.ensemble import RandomForestClassifierimport joblibedge_model = RandomForestClassifier(n_estimators=20, max_depth=8, random_state=42)edge_model.fit(X_train, y_train)joblib.dump(edge_model, "edge_model.pkl")print("Compressed model saved")

Compare model size

import osprint("Edge model size:", round(os.path.getsize("edge_model.pkl") / 1024, 2), "KB")

Measure prediction latency

import timeimport numpy as nptimes = []for i in range(len(X_test)): start = time.perf_counter() edge_model.predict(X_test.iloc[[i]]) end = time.perf_counter() times.append((end - start) * 1000)times.sort()mean_latency = np.mean(times)p95_latency = np.percentile(times, 95)print("Mean latency:", mean_latency, "ms")print("95th percentile:", p95_latency, "ms")print("Throughput:", 1000 / mean_latency, "samples/sec")

Comparison table

Measurement	**Full model**	**Edge model**
Number of trees	100	20
Maximum depth	Default	8
Model size	Measure	Measure
Accuracy	Measure	Measure
F1-score	Measure	Measure
Mean latency	Measure	Measure
95th percentile latency	Measure	Measure
Throughput	Measure	Measure

Decision: The edge model is preferable when it meets the required accuracy and 50 ms latency budget. If its false-negative rate becomes too high, the system should send uncertain cases to a more powerful cloud model.

TASK B4 – Final Engineering Decision: Conversational Interface and Deployment (5 Marks)

B4.1 – Implement a Practical Retrieval-Based Chatbot

The chatbot allows operators to ask questions about intrusion alerts.

Eight intents

Intent	Example question
Greeting	Hello
Open alerts	Which alerts are open?
Alert details	Show alert details
Attack type	What type of attack was detected?
Normal traffic	Is this traffic normal?
Model accuracy	What is the model accuracy?
Explanation	Why was this flagged?
Help	What can you do?

Python chatbot code

Install the required library:

pip install scikit-learn

Program:

from sklearn.feature_extraction.text import TfidfVectorizerfrom sklearn.metrics.pairwise import cosine_similarityquestions = ["hello hi good morning", "which alerts are open show alerts", "show alert details explain alert", "what type of attack was detected attack type", "is traffic normal check network traffic", "what is model accuracy show accuracy", "why was this flagged explain prediction", "help what can you do"]answers = ["Hello! How can I help you?", "You can view the currently open security alerts.", "Alert details include time, type and severity.", "The system identifies the possible attack category.", "The model checks whether network traffic is normal or malicious.", "Model accuracy is available in the evaluation report.", "The alert was generated because the input matched suspicious patterns.", "I can help with alerts, attacks, accuracy and model explanations."]vectorizer = TfidfVectorizer()vectors = vectorizer.fit_transform(questions)threshold = 0.25while True: user = input("nYou: ") if user.lower() == "exit": break

Sample input and expected output

You: Which alerts are open?

Bot: You can view the currently open security alerts.

You: Why was this flagged?

Bot: The alert was generated because the input matched suspicious patterns.

You: What is the weather?

Bot: Sorry, I cannot understand. Please ask another question.

Evaluation

Use labelled test questions for all eight intents. Calculate precision, recall and F1 for each intent using the predicted intent and actual intent.

[Precision = frac{TP}{TP+FP}]

[Recall = frac{TP}{TP+FN}]

[F1 = frac{2PR}{P+R}]

The confusion matrix shows which intents are incorrectly classified as other intents.

Response latency: Measure the time between receiving a question and displaying the chatbot response. Report the mean and 95th percentile latency.

B4.2 – Simulate Speech-Recognition Errors

Speech-recognition errors can replace, delete or insert words.

Example:

Word error rate	Original question	Corrupted question
0%	Which alerts are open?	Which alerts are open?
10%	Which alerts are open?	Which alerts open?
20%	Which alerts are open?	Which alert open?
30%	Which alerts are open?	Which open?

Expected result

Word error rate	Expected intent accuracy
0%	Highest
10%	Slight reduction
20%	Further reduction

30%	Greater reduction

As word errors increase, the chatbot may fail to identify the correct intent.

Threshold-sweep experiment

Test different confidence thresholds such as:

0.10, 0.20, 0.30, 0.40, 0.50

For each threshold, calculate:

- Intent accuracy.
- Out-of-scope detection rate.
- False-accept rate.

A low threshold accepts more questions but may accept unrelated questions. A high threshold rejects more unrelated questions but may also reject valid questions.

Conclusion: A suitable threshold balances correct intent recognition and rejection of unrelated questions.

B4.3 – Production Decision

The system contains two main components:

1. Intrusion detection model.
2. Conversational assistant.

Deployment comparison

Factor	**Edge deployment**	**Cloud deployment**
Latency	Usually lower for local inference	Depends on network connection
Privacy	Network data can remain local	Data may need to be transmitted
Processing power	Limited	Higher computing resources
Cost	Device and maintenance cost	Server and network cost
Availability	Can work without internet	Requires connectivity for cloud services

Model updates	More difficult	Easier to update centrally

My deployment decision

I recommend a hybrid architecture.

- Run the compressed intrusion detection model on the edge device for immediate detection.
- Send uncertain or high-risk cases to the cloud for detailed analysis.
- Run the chatbot on the edge for basic questions and offline access.
- Use the cloud for advanced reports, centralised monitoring and model updates.

Reasons:

1. Edge processing reduces dependence on internet connectivity.
2. Sensitive network information can remain on the local device.
3. Cloud processing provides more computing power.
4. The hybrid approach balances response time, privacy and cost.
5. The edge model can continue basic detection during network failure.

When would I change this decision?

If the edge device has insufficient memory or processing power, I would move more processing to the cloud. If internet connectivity becomes unreliable or data privacy requirements become stricter, I would move more functions to the edge.

Final conclusion: A hybrid deployment is suitable because it combines fast local detection with powerful cloud-based analysis.

Overall Conclusion

Artificial Intelligence systems require more than a working algorithm. They must be designed, implemented, tested and evaluated under real-world limitations.

In this assignment:

- A hybrid drone agent provides safe parcel delivery.
- A decentralised swarm improves search-and-rescue coverage.
- An expert-system shell provides reusable rule-based reasoning.
- PySpark RDDs process large sensor datasets.
- Model compression makes AI suitable for edge devices.
- A retrieval-based chatbot provides a conversational interface.

- A hybrid deployment balances latency, privacy, computing power and cost.

Thus, an AI engineer must consider accuracy, safety, memory, processing time, scalability and reliability before deploying an intelligent system.

References

1. Russell, S. and Norvig, P., Artificial Intelligence: A Modern Approach, Pearson.
2. Apache Spark Documentation – RDD Programming Guide: https://spark.apache.org/docs/latest/rdd-programming-guide.html
3. Scikit-learn Documentation – Machine Learning: https://scikit-learn.org/stable/
4. UNSW Canberra – UNSW-NB15 Network Intrusion Dataset: https://research.unsw.edu.au/projects/unsw-nb15-dataset
5. Python Documentation: https://docs.python.org/3/